

Pollinexus Notebook — Hands-on Validation Guide (pollinexus-done.ipynb)

Intro Speech (Data Science + Programming Understanding)

As a data scientist and software engineer, I approach this project end-to-end: from rigorous data quality assessment and domain-driven cleaning to explainable machine learning, reproducible visualization, and production-grade APIs. I translate ecological questions into measurable targets, validate with statistics, and codify workflows with clean, modular Python. I emphasize reliability (error handling, logging), security (sanitization, least privilege), and observability (metrics, health checks). The goal is actionable insight that scales from a notebook into an API.

0) Environment setup

I'll ensure the environment is ready and the dataset is present before execution.

- Python 3.12+
- Install project in editable mode:

```
pip install -e .
```

- Ensure dataset exists:
 - Expected path: `dataset/plants_and_bees.csv`

1) Open the notebook

I open the curated notebook and confirm the kernel to guarantee consistent dependencies.

- File: `src/pollinexus/notebook/pollinexus-done.ipynb`
- Kernel: Python 3 (ipykernel)

2) Run sections in order

I execute top-to-bottom, validating outputs after each block to maintain a reliable analytical chain.

2.1 Setup, Data Loading, and System Monitoring

I import libraries, load the CSV with robust path handling, and inspect the dataset footprint.

- Import libraries; expect confirmation prints
- Load CSV via robust path resolution (multiple fallbacks)
- Display `head()`, `info()`, `describe()`
- Validate output:
 - Shape ~ (1250, 16)

- Columns printed, head rendered, info and stats displayed

2.2 Data Quality Assessment

I quantify missingness and visualize it to guide informed cleaning decisions.

- Compute missing counts and percentages
- Render bar chart of missing percentages
- Validate output:
 - Columns with missing values are listed
 - Visualization figure renders without errors

2.3 Data Cleaning and Preprocessing

I apply domain-driven rules (e.g., Air_Sampling) and engineer features needed for downstream analysis.

- Convert types (date → datetime, binary → int)
- Fill domain-informed defaults (e.g., plant_species → Air_Sampling)
- Feature engineering: native_bee, month, time_of_day
- Validation prints: cleaned shape, missing count = 0 (or expected minimal)

2.4 Exploratory Data Analysis (EDA)

I establish core distributions and coverage patterns to frame hypotheses and expectations.

- Dataset overview: counts, unique categories, time span
- Native vs Non-native distribution
- Plant species analysis (excluding Air_Sampling)
- Seasonal pattern summary
- Sampling method effectiveness (diversity per record)
- Validate numeric summaries match expectations (native-dominant system)

2.5 Data Visualizations

I render a compact dashboard to communicate patterns at a glance.

- Dashboard with 4 plots:
 - Top plants by visits
 - Native vs Non-native pie
 - Diversity by sampling method
 - Seasonal activity
- Validate: Figure renders, axes/titles correct, no exceptions

2.6 Machine Learning Analysis

I prepare features, encode categories, and train an interpretable model, emphasizing feature importance.

- Prepare ML dataset (exclude Air_Sampling)
- Encode categorical features with LabelEncoder

- Train/test split (80/20, stratified)
- RandomForestClassifier (class_weight balanced)
- Report accuracy and classification_report
- Feature importance ranking (expect Plant Species dominant)
- Validate: Accuracy $\sim 0.8 \pm$ (indicative), importances printed and reasonable

2.7 Plant Species Analysis and Ranking

I compute a balanced composite score (visit/native/abundance) to rank practical choices.

- Aggregate metrics per plant: visits, native rate, abundance, diversity
- Compute normalized scores and weighted composite score (40/40/20)
- Display top-10 by composite score
- Validate: Rankings printed with composite scores in [0,1]

2.8 Top 3 Plant Recommendations

I present early/mid/late-season selections with metrics and rationale for a season-long plan.

- Prints three detailed recommendations (early, mid, late season)
- Includes performance metrics and rationale
- Validate: Coverage plan (50/30/20) printed and consistent with previous section

2.9 Seasonal Coverage Analysis

I validate seasonality and ensure no gaps in support across the active months.

- Crosstab for top plants $\times$ season
- Bar charts and planting calendar summary
- Validate: Early/late counts reasonable; timeline printed

2.10 Conclusions and Strategic Recommendations

I summarize findings, model performance, and implementation actions, then export artifacts.

- Summarize key findings, ML accuracy, importance, plant strategy
- Export `plant_recommendations_analysis.csv`
- Validate: CSV created in notebook working directory

3) Troubleshooting checklist

If something fails, I verify paths, kernel state, and plotting backends before re-running.

- File paths: ensure `dataset/plants_and_bees.csv` exists
- Kernel: restart and run all if state is stale
- Dependencies: install `matplotlib`, `seaborn`, `scikit-learn`, `pandas`
- Plots not visible: ensure `%matplotlib inline` (if needed) and no backend errors

4) Mapping to API

I map each notebook step to the API so stakeholders can reproduce results via services.

- The notebook flow aligns with API groups:
 - Dataset → upload/info/health
 - Analysis → jobs/status/results
 - Visualizations → generation/status/download
- Use [docs/API.md](#) for curl-based API testing